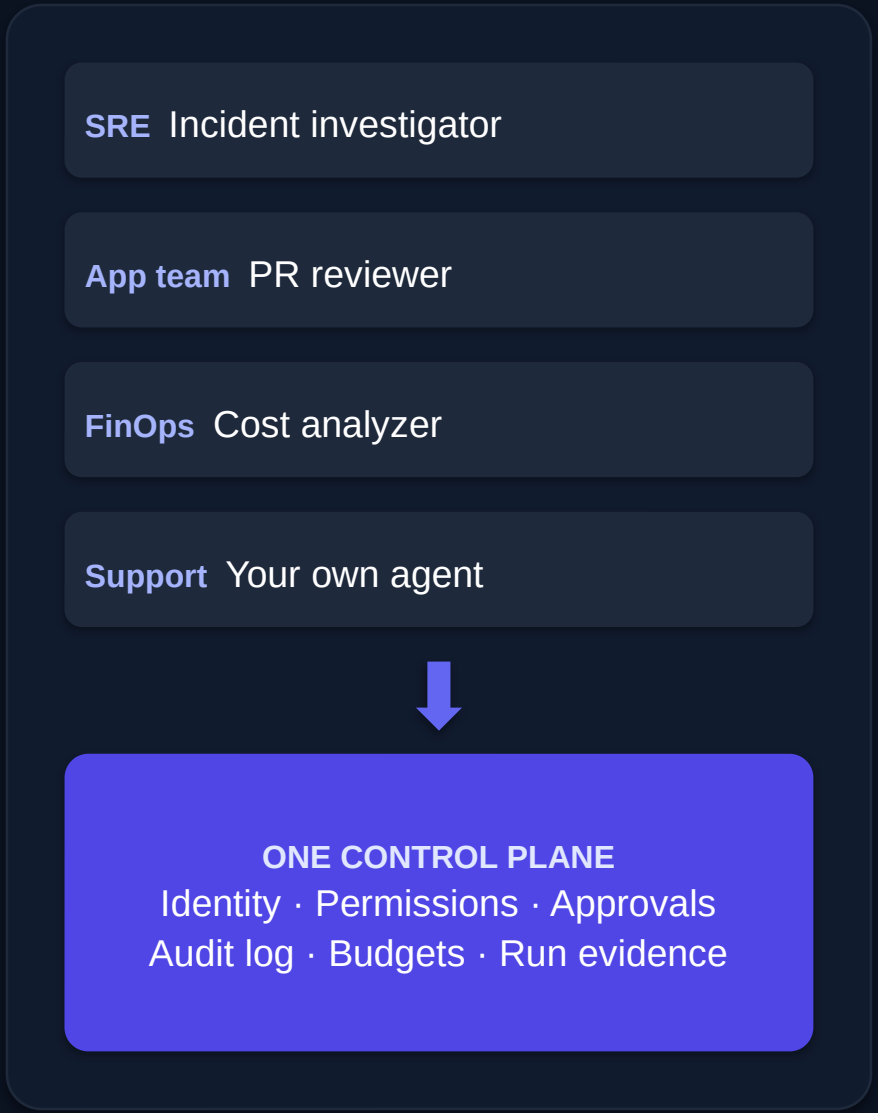
KOMODOR AGENTIC OPERATIONS PLATFORM

Your teams are already shipping agents.

Can you govern them?

Give every agent an identity, a boundary, an approval path and a record.



A conversation for platform engineering leaders

Agents are spreading faster than your operating model

Every team's agent is a good idea. The risk sits between them.

NO INVENTORY

Who owns it?

Agents appear team by team. Nobody has the full list.

INCONSISTENT ACCESS

What can it touch?

Each agent brings its own keys. Its limits live in a prompt.

NO SINGLE RECORD

What did it do?

Alert, investigation and fix sit in three different tools.

40%+ of agentic AI projects will be cancelled by 2027, says Gartner, for governance, unclear value and cost.

Cited in Komodor's KAOP launch, September 2026

ASK Which of these three hurts your team most today?

KAOP runs agents like production services

One platform for Komodor's agents and your own.

BUILD

Start ready, or bring your own

Komodor-built agents, including Kubernetes RCA from Klaudia. Or import yours.

RUN

Real triggers, real workflows

Alerts, schedules and Slack start runs. Workflows route to specialists.

GOVERN

Enforced, not instructed

Identity, default-deny permissions, guardrails, approvals and audit.

OPTIMIZE

Keep what earns its cost

Every run's evidence and cost. Evaluations and budgets.

FOUNDATION Klaudia's Kubernetes expertise, packaged as agents your teams deploy in minutes.

A prompt is not a permission model

KAOP governs an agent the way it governs a person.

	LAYER	ASKS	STOPS
1	**Identity**	Who is acting?	Unaccountable actions
2	**Permissions**	Allowed on this?	Reaching beyond the job
3	**Standards**	Built to our rules?	Agents with no owner or budget
4	**Guardrails**	Safe right now?	Destructive tool calls
5	**Approvals**	Should a person decide?	Irreversible actions alone
6	**Audit log**	What happened?	Changes nobody can explain

IN TODAY'S LAB

Investigator
Reads only. No secrets, no writes.

GitHub agent
One PR, after I approve. Never merges.

Code reviewer
Comments only. Can't approve or merge.

PRINCIPLE If I say an agent is read-only, I show what enforces it.

One record from the alert to the merged fix

What the platform team sees, without asking anyone.

Fleet health

Every agent, its owner, its status.

Runs & evidence

Inputs, tool calls, reasoning, output, cost.

Attention

Decisions waiting on a person.

Audit log

Who changed what, and when.

Spend

Cost per agent and team, with budgets.

Access explorer

What can this agent actually do?

RESULT Inventory, access and audit become a screen, not a spreadsheet.

One incident, two governed workflows

Built from KAOP catalog agents. No custom code.

WORKFLOW 1 · INCIDENT · owned by SRE · starts on a Grafana alert



WORKFLOW 2 · CODE REVIEW · owned by the app team · starts on any pull request



NOTICE No agent changes the running cluster. Fixes go through git and a person.

Who can do what, and what enforces it

Least privilege per agent, enforced outside the agent.

Agent	Can read	Can change	Enforced by
Kubernetes RCA	Pods, events, logs, workloads	Nothing	Kubernetes RBAC on its own identity
Grafana investigator	Metrics, events, alerts	Nothing	Grafana Viewer token
Remediation	The investigation	Nothing	Holds no credentials
GitHub agent	Repository code	One PR, after approval	Approval gate + GitHub app scopes
Code reviewer	The PR diff	One comment	GitHub app scopes

VERIFIED ON THIS CLUSTER

KAOP's agent permissions checked with kubectl: reads pods, events, logs. Denied secrets, exec, delete, patch, scale.

Live: one bad change, one Redis outage

Real cluster, real alert, real agents.

BREAK

1

Bad change

Liveness checks Redis.

2

Redis goes down

Readiness fails. Correct.

3

Amplified

Kubernetes kills the healthy API.

INVESTIGATE

4

Detect

Grafana alerts KAOP.

5

Investigate

Root cause, confidence, evidence.

FIX

6

Decide

I approve a one-line PR.

7

Review

A second agent reviews it.

MEASURED IN REHEARSAL

Bad check API killed 5 times in 5 minutes, alert fired

Fixed check 0 kills, alert stayed quiet

WATCH FOR Redis is the trigger. The health check is the root cause.

Start with one owned workflow

Two weeks, one owner, pass criteria agreed up front.

WEEK 1

Incident investigation

One cluster. Read-only. Approval on every write.

WEEK 2

Add code review

One repository. Fixes go through your normal merge.

THEN

Scale or stop

Review the scorecard together. Expand only if it holds.

PILOT SCORECARD

Evidence

Every RCA cites real evidence

Useful

On-call would act on it

In policy

Zero unapproved writes

Cost

Within the agreed budget

ASK Which workflow would you start with, and who would own it?

How I built this pitch

Every claim traces to Komodor's material or to this lab.

Komodor website and KAOP launch	Positioning: build, run, govern, optimize. The Gartner figure.
KAOP docs: security and governance	The six layers, approvals, audit log, Access explorer.
KAOP docs: agent catalog	What each agent reads, changes, and where it stops.
KAOP docs: incident module	The alert-to-remediation workflow.
This lab	A real fault, a real alert, permissions verified, fix measured.

STRATEGY Lead with their pain, prove it with one incident, close on a small pilot.

Questions I expect

“We'll build this ourselves.”	You can build an agent. The hard part is identity, approvals, audit, budgets and evaluation around every agent, for every team. That's the platform.
“Komodor already has Klaudia. Why KAOP?”	Klaudia is the expertise. KAOP is the operating model that runs it beside your own agents, under one set of rules.
“Do we still need Grafana?”	Yes. KAOP doesn't replace your monitoring. It starts where your alerts already are, and its Grafana agent reads the same data.
“What if the agent is wrong?”	It separates what it saw from what it guessed, every change needs a person, and KAOP grades runs over time.
“How do you know it's the probe?”	I ran the same outage both ways. With the bad check, Kubernetes killed the API five times in five minutes and the alert fired. With the fix: zero kills, no alert.

Rehearsal evidence

The same Redis outage, measured both ways.

	Liveness checks Redis (bad)	Liveness checks the process (fix)
Times Kubernetes killed the healthy API	5 in a 5-minute outage	0
Liveness failures	Yes	None
Grafana restart alert	Fired at +3 minutes	Stayed Normal
API back after Redis returned	4.8 s (between back-offs)	6.5 s

Rehearsal on 2026-10-24, single-node kind cluster. With the bad check, recovery time depends on where the restart back-off is when Redis returns (10 s, doubling up to 5 minutes).